



Material do Estudante

Programação Assistida e Automação com Inteligência Artificial

Disciplina: Tendências em Ciência da Computação

Unidade II - Programação Assistida

Conteúdos integrados: 17/09 e 24/09 | Duração: 1h30

Professora Kadidja Valéria

Objetivo do material: apoiar o estudo, a prática em laboratório e o registro da atividade no GitHub.

Disciplina: Tendências em Ciência da Computação

Unidade: II — Programação Assistida

Conteúdos integrados: 17/09 e 24/09

Duração: 1h30

Tema: Programação Assistida por IA e Automação com IA

Professora: Kadidja Valéria

1. Objetivos de Aprendizagem

Ao final da aula, você deverá ser capaz de:

- compreender o conceito de **programação assistida por IA**;
- reconhecer usos da IA para geração, explicação, correção e refatoração de código;
- elaborar prompts para tarefas de programação;
- compreender o conceito de automação com scripts;
- identificar tarefas adequadas à automação;
- construir e testar um script simples com apoio de IA;
- analisar criticamente código gerado por IA;
- validar, corrigir e refatorar uma solução antes de utilizá-la;
- reconhecer aspectos de segurança, ética e responsabilidade no uso de IA em programação.

2. Questão Norteadora

Como utilizar Inteligência Artificial para apoiar o desenvolvimento, aprimoramento e automação de soluções computacionais sem abrir mão da compreensão, validação e responsabilidade sobre o código produzido?

A aula integra dois movimentos:

```
PROGRAMAÇÃO ASSISTIDA
↓
Gerar • Explicar • Corrigir • Refatorar
↓
AUTOMAÇÃO
↓
Transformar tarefas repetitivas em scripts
↓
TESTAR
↓
VALIDAR
↓
MELHORAR
```

PARTE I — PROGRAMAÇÃO ASSISTIDA POR IA

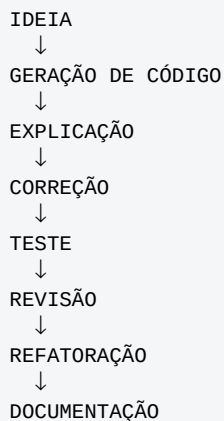
3. O que é Programação Assistida por IA?

Programação assistida por IA é o uso de sistemas de Inteligência Artificial como apoio durante atividades relacionadas ao desenvolvimento de software.

A IA pode apoiar tarefas como:

- geração de código;
- explicação de trechos de código;
- identificação de erros;
- depuração;
- criação de testes;
- refatoração;
- documentação;
- sugestão de alternativas de implementação.

Fluxo típico:

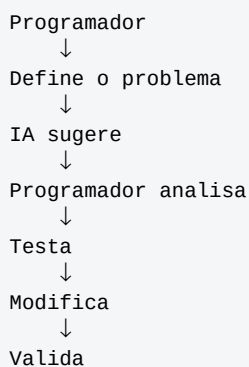


4. IA como apoio, não como substituição

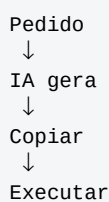
Uma ferramenta de IA pode produzir código rapidamente, mas isso não significa que o código esteja correto, seguro, eficiente ou adequado ao problema.

Compare os dois fluxos:

Uso assistido



Uso passivo



O primeiro fluxo favorece aprendizagem, compreensão e responsabilidade. O segundo aumenta o risco de erros e uso indevido.

5. Geração de Código com IA

Um prompt genérico pode produzir uma solução pouco adequada.

Exemplo pouco específico

```
Faça um programa em Python para números pares.
```

Exemplo estruturado

```
Crie uma função em Python que receba uma lista  
com números inteiros e retorne somente os números pares.
```

```
Inclua:  
- type hints;  
- docstring;  
- um exemplo de uso.
```

```
Não utilize bibliotecas externas.
```

O que melhorou?

O segundo prompt define:

- linguagem;
- entrada;
- objetivo;
- restrições;
- formato esperado.

6. Explicação de Código com IA

A IA pode ser utilizada como tutora para explicar código.

Prompt sugerido

```
Explique o código abaixo para um estudante iniciante  
de Ciência da Computação.
```

```
Explique linha por linha e, ao final, descreva  
o objetivo geral do algoritmo.
```

```
[CÓDIGO]
```

Ao utilizar esse tipo de prompt, tente responder depois, com suas próprias palavras:

1. Qual é a entrada do programa?
2. Qual processamento ele realiza?
3. Qual é a saída?
4. Qual parte você ainda não compreende?

7. Identificação e Correção de Erros

Considere o código:

```
numeros = [10, 15, 21, 30, 42]  
  
for numero in numeros:  
    if numero % 2 = 0:
```

```
print(numero)
```

Há um erro de sintaxe na condição.

Em vez de simplesmente pedir:

```
Corrija meu código.
```

Prefira:

```
Analise o código abaixo.
```

1. Identifique os erros.
2. Explique por que cada erro ocorre.
3. Informe o conceito relacionado ao erro.
4. Sugira uma correção.
5. Só depois apresente o código completo corrigido.

```
[CÓDIGO]
```

O objetivo é compreender o problema e não apenas receber a resposta pronta.

8. Depuração com IA

Considere a função:

```
def media(notas):  
    soma = 0  
  
    for nota in notas:  
        soma += nota  
  
    return soma / len(nota)
```

O problema está na última linha: `nota` representa apenas o último valor percorrido no laço, enquanto a intenção é utilizar a quantidade de elementos da lista `notas`.

Prompt para análise

```
Analise a função Python abaixo.
```

```
Não corrija imediatamente.
```

```
Primeiro:
```

1. identifique o problema;
2. explique por que ele acontece;
3. indique qual conceito de Python está relacionado ao erro;
4. depois apresente uma versão corrigida.

```
[CÓDIGO]
```

9. Refatoração de Código

Refatorar significa melhorar a estrutura interna de um código sem alterar seu comportamento esperado.

A refatoração pode buscar:

- maior legibilidade;
- redução de repetição;
- melhores nomes de variáveis;

- melhor divisão em funções;
- tratamento de erros;
- melhor documentação;
- menor complexidade desnecessária.

Código inicial

```
lista = [1, 2, 3, 4, 5, 6]
nova = []

for x in lista:
    if x % 2 == 0:
        nova.append(x)

print(nova)
```

Possível versão refatorada

```
numeros = [1, 2, 3, 4, 5, 6]
pares = [numero for numero in numeros if numero % 2 == 0]

print(pares)
```

A segunda versão é mais curta, mas isso não significa que seja sempre melhor. A qualidade também depende do público, da manutenção, da clareza e do contexto de uso.

Prompt para refatoração

```
Atue como revisor de código Python.

Analise o código abaixo considerando:
- legibilidade;
- nomes de variáveis;
- duplicações;
- modularização;
- tratamento de erros.

Não altere o comportamento da aplicação.

Apresente:
1. problemas encontrados;
2. sugestões de melhoria;
3. código refatorado;
4. justificativa das principais alterações.

[CÓDIGO]
```

PARTE II — AUTOMAÇÃO COM IA

10. O que é Automação?

Automação é a utilização de software para executar tarefas com pouca ou nenhuma intervenção manual.

Exemplos de tarefas que podem ser automatizadas:

- organizar arquivos;
- renomear documentos;
- processar planilhas;
- validar dados;
- converter formatos;

- criar relatórios;
- analisar logs;
- realizar cálculos repetitivos;
- gerar resumos de dados;
- realizar backups ou verificações periódicas.

11. Quando uma tarefa é boa candidata à automação?

Uma tarefa tende a ser adequada quando é:

- repetitiva;
- frequente;
- baseada em regras claras;
- previsível;
- sujeita a erros manuais;
- realizada sobre grande quantidade de dados.

Pergunta para reflexão

Automatizar sempre é a melhor opção?

Não. Se a tarefa é rara, pouco previsível ou exige julgamento humano complexo, a automação pode não compensar o esforço ou pode aumentar riscos.

12. Exemplo de Problema de Automação

Imagine a seguinte pasta:

```
trabalhos/
■■■ Relatorio João FINAL.pdf
■■■ relatorio_maria.PDF
■■■ trabalho-pedro-final.pdf
■■■ Relatorio Ana.pdf
```

Deseja-se padronizar os nomes:

```
trabalhos/
■■■ joao_relatorio.pdf
■■■ maria_relatorio.pdf
■■■ pedro_relatorio.pdf
■■■ ana_relatorio.pdf
```

Essa tarefa pode ser automatizada com um script Python.

Antes de escrever o código, é necessário definir:

- quais arquivos serão alterados;
- qual regra de nomeação será usada;
- como evitar sobrescrever arquivos;
- como tratar nomes inesperados;
- se deve haver uma simulação antes da alteração real.

13. Engenharia de Prompt aplicada à Programação

Um prompt para programação pode ser organizado com a estrutura:

PAPEL
+
PROBLEMA
+
ENTRADA
+
SAÍDA ESPERADA
+
TECNOLOGIA
+
RESTRIÇÕES
+
CASOS DE TESTE
+
CRITÉRIOS DE QUALIDADE

Prompt pouco específico

Faça um programa para organizar arquivos.

Prompt estruturado

Atue como desenvolvedor Python.

PROBLEMA:
Preciso organizar automaticamente arquivos PDF existentes em uma pasta.

TAREFA:
Crie um script que liste todos os arquivos .pdf e apresente os nomes encontrados.

RESTRIÇÕES:
- utilizar Python;
- utilizar somente a biblioteca padrão;
- não excluir nenhum arquivo;
- não renomear arquivos nesta primeira versão.

FORMATO:
1. explique a estratégia;
2. apresente o código;
3. explique as principais linhas;
4. apresente um exemplo de execução.

14. Desenvolvimento Incremental

Evite pedir uma solução complexa de uma única vez.

Uma estratégia mais segura é desenvolver por etapas:

```
ETAPA 1
Identificar arquivos
    ↓
ETAPA 2
Validar regras
    ↓
ETAPA 3
Simular modificações
    ↓
ETAPA 4
Executar modificações
    ↓
ETAPA 5
Tratar erros
    ↓
ETAPA 6
Registrar resultados
```


Essa abordagem facilita testes, revisão e correção de problemas.

PARTE III — ATIVIDADE PRÁTICA

15. Desafio Hands On — Programação Assistida e Automação com IA

Organização: individual ou em dupla

Valor: 0,5 ponto

Objetivo

Identificar uma tarefa computacional simples e repetitiva e construir um script Python para automatizá-la utilizando uma ferramenta de IA como apoio.

16. Escolha um Problema

Opção A — Organização de Arquivos

Crie um programa que identifique arquivos em uma pasta e os organize por extensão.

Opção B — Processamento de Notas

Crie um programa que receba uma lista de notas e apresente:

- média;
- maior nota;
- menor nota;
- quantidade de estudantes aprovados.

Opção C — Análise de Logs

Considere entradas como:

```
INFO - usuário autenticado
ERROR - conexão perdida
INFO - arquivo salvo
WARNING - espaço reduzido
ERROR - banco indisponível
```

Crie um script que conte quantas ocorrências existem de:

- `INFO`;
- `WARNING`;
- `ERROR`.

Opção D — Gerador de Relatório

Crie um programa que receba valores de vendas e produza um resumo com:

- total;
- média;
- maior valor;
- menor valor.

Opção E — Problema proposto pelo estudante

Você pode propor outro problema simples de automação, desde que seja possível explicar claramente a entrada, o processamento e a saída esperada.

17. Etapa 1 — Planeje antes de usar IA

Preencha:

```
## Problema

Qual tarefa quero automatizar?

## Entrada

Quais dados o programa receberá?

## Processamento

O que o programa deverá fazer?

## Saída

Qual resultado espero obter?
```

18. Etapa 2 — Construa o Prompt

Utilize como referência:

```
PAPEL:

CONTEXTO:

PROBLEMA:

ENTRADA:

SAÍDA ESPERADA:

LINGUAGEM:

RESTRICÇÕES:

CRITÉRIOS DE QUALIDADE:

CASOS DE TESTE:
```

19. Etapa 3 — Analise o Código Gerado

Antes de executar, verifique:

- ☐ Eu compreendo o código?
- ☐ O código atende ao problema definido?
- ☐ Há bibliotecas que eu não conheço?
- ☐ Há operações que podem apagar ou sobrescrever dados?
- ☐ O código utiliza dados sensíveis?
- ☐ Há tratamento de erros?
- ☐ Consigo explicar cada função ou bloco principal?
- ☐ Existem casos que o código não considera?

20. Etapa 4 — Teste a Solução

Crie pelo menos três casos de teste.

Teste 1 — Caso normal

Entrada válida
→ resultado esperado

Teste 2 — Caso limite

Entrada vazia, lista pequena ou valor extremo
→ comportamento esperado

Teste 3 — Caso de erro

Entrada inválida
→ tratamento esperado

Registre o que aconteceu em cada teste.

21. Etapa 5 — Solicite Refatoração

Depois que o programa funcionar, utilize um prompt semelhante a:

Revise o código abaixo.

O programa já funciona.

Agora analise:
- clareza;
- organização;
- duplicação;
- nomes de variáveis e funções;
- tratamento de erros.

Sugira melhorias sem alterar o comportamento esperado.
Explique cada alteração proposta.

[CÓDIGO]

22. Etapa 6 — Compare as Versões

Atribua uma nota de 1 a 5 para cada critério.

Critério	Versão inicial	Versão refatorada
Funcionamento correto		
Clareza		
Organização		
Legibilidade		
Tratamento de erros		
Facilidade de manutenção		

PARTE IV — SEGURANÇA, ÉTICA E RESPONSABILIDADE

23. Responsabilidade sobre o Código

A pergunta mais importante não é:

“Foi a IA que escreveu?”

Mas sim:

“Você compreende, testou e consegue defender tecnicamente o código que está utilizando?”

Uma IA pode sugerir:

- código incorreto;
- bibliotecas inadequadas;
- APIs inexistentes;
- soluções inseguras;
- código desatualizado;
- soluções excessivamente complexas.

24. Regra do Desenvolvedor

```
IA SUGERE
  ↓
EU ANALISO
  ↓
EU TESTO
  ↓
EU MODIFICO
  ↓
EU VALIDO
  ↓
EU ASSUMO RESPONSABILIDADE
```

25. Segurança ao compartilhar informações com IA

Evite inserir em prompts:

- senhas;
- chaves de API;
- tokens;
- dados pessoais;
- dados confidenciais;
- credenciais de banco de dados;
- código proprietário que não pode ser compartilhado.

Evite

```
API_KEY = "minha-chave-real"
PASSWORD = "senha123"
```

Prefira exemplos fictícios

```
API_KEY = "<SUA_CHAVE>"
PASSWORD = "<SENHA>"
```

PARTE V — REFLEXÃO E ENTREGA

26. Take Away

Responda individualmente:

1. Em qual etapa a IA foi mais útil: geração, explicação, depuração, testes, refatoração ou documentação? Justifique.
2. Qual parte da solução exigiu mais raciocínio humano?
3. A primeira solução gerada funcionou? Se não, qual foi o problema?
4. O que mudou após a refatoração?
5. Você conseguiria explicar o programa sem consultar a resposta da IA?
6. Qual foi o principal risco identificado no uso da IA durante a atividade?

27. Desafio Final

Complete a frase:

“Programar com IA não significa deixar a IA programar por mim. Significa...”

Depois, formule cinco regras para um desenvolvedor utilizar IA de maneira responsável:

- 1.
- 2.
- 3.
- 4.
- 5.

28. Produto Final da Atividade

Entregue:

- descrição do problema;
- entrada, processamento e saída esperada;
- prompt utilizado;
- primeira versão do código;
- análise crítica da primeira versão;
- pelo menos três casos de teste;
- registro dos resultados dos testes;
- prompt utilizado para refatoração;
- versão refatorada do código;
- comparação entre as versões;
- reflexão final.

29. Modelo de Registro no GitHub

Nome sugerido do arquivo:

programacao-assistida-automacao-ia.md

Estrutura sugerida:

```
# Programação Assistida e Automação com IA

## Identificação

- Nome:
- Turma:
- Data:
- Ferramenta de IA utilizada:

## 1. Problema

...

## 2. Entrada

...

## 3. Processamento

...

## 4. Saída esperada

...

## 5. Prompt utilizado

...

## 6. Código inicial
```

código

```
## 7. Análise crítica

...

## 8. Casos de teste

### Teste 1

...

### Teste 2

...

### Teste 3

...

## 9. Problemas encontrados

...

## 10. Prompt de refatoração

...

## 11. Código refatorado
```

código

```
## 12. Comparação

| Critério | Inicial | Refatorado |
|---|---|---|
```

```
| Funcionamento | | |  
| Clareza | | |  
| Organização | | |  
| Legibilidade | | |  
| Tratamento de erros | | |
```

13. Reflexão

- Onde a IA mais ajudou?
- Onde a IA errou?
- O que precisei modificar?
- Consigo explicar o código?

14. Take Away

Programar com IA não significa...

30. Critérios de Avaliação — 0,5 ponto

Critério	Valor
Definição adequada do problema	0,05
Construção do prompt	0,10
Desenvolvimento da automação	0,10
Testes e validação	0,10
Refatoração e comparação	0,05
Análise crítica do uso da IA	0,05
Organização do registro no GitHub	0,05
Total	**0,50**

31. Checklist de Entrega

- ☐ Defini o problema.
- ☐ Identifiquei entrada, processamento e saída.
- ☐ Registre o prompt utilizado.
- ☐ Registre a primeira versão do código.
- ☐ Analisei criticamente a solução.
- ☐ Executei pelo menos três testes.
- ☐ Registre os resultados.
- ☐ Solicitei e analisei uma refatoração.
- ☐ Comparei as versões.
- ☐ Respondi à reflexão final.
- ☐ Organizei o arquivo no GitHub.

32. Síntese da Aula

```
PROBLEMA  
↓  
DEFINIR REQUISITOS  
↓  
CRIAR PROMPT  
↓  
IA GERA UMA SUGESTÃO
```

↓
COMPREENDER O CÓDIGO
↓
TESTAR
↓
CORRIGIR
↓
REFATORAR
↓
VALIDAR
↓
DOCUMENTAR

□ Take Away da Aula

A IA pode acelerar a escrita do código, mas a qualidade do software continua dependendo da capacidade humana de compreender o problema, analisar a solução, testar resultados e tomar decisões.